Why tbb::fixed_pool cannot use the whole arena

Huge-object size rounding in tbbmalloc, and what it means for large Fortran arrays

Summary

There is an explanation for the loss of usable memory observed with **tbb::fixed_pool**, but it is in the source code of *tbbmalloc*, not in the API documentation. For large blocks the requested size is rounded up to a geometric grid, so the gap is proportional to the size of the arena: at worst one eighth (12.5%) of a block, about 6% on average, and about 6.25% for power-of-two arenas.

What the documentation says

The reference page of **fixed_pool** only says that all the memory available for allocation is passed through the constructor and that allocation scales with the number of processors. It does not mention any overhead. Intel's article on the oneTBB allocator explains that large objects are cached in two families of bins, "large" and "huge", which differ only in the size of their bins. This is the lead to follow.

What the source code says

The following was read in oneTBB 2021.5.0 (*backend.cpp*, *backend.h*, *large_objects.h*, through the copy published by the *tbb-sys* crate). Your version may differ, but this mechanism has existed for a long time.

- The buffer of a fixed_pool is requested **once and only once**; any later request returns NULL. The buffer therefore becomes a single large region, which is then split into blocks.
- For objects larger than 8 MiB, the requested size is **rounded to a geometric grid**: there are 8 bins between two powers of 2, and the size is rounded up to the next multiple of 2^(floor(log2(size)) - 3). The step is therefore one eighth of the power of 2 just below the requested size.

Application to your figures

Arena: 512 MiB. Two arrays of 251,658,240 bytes (exactly 240 MiB) are refused by tbb, whereas the double stack accepts two arrays of 256 MiB.

For a 240 MiB array, the power of 2 just below is 128 MiB, so the step is 16 MiB. The 240 MiB requested, plus headers, slightly exceeds 240 MiB exactly and is rounded up to **256 MiB**. Two such blocks occupy 512 MiB, the whole buffer, while the region header and the end-of-region markers also have to fit inside it. The second allocation therefore fails, exactly as observed.

Consequence: a proportional gap

For an arena of size 2^k, the largest allocatable block should be 15/16 of the arena (6.25% loss). This can be tested with the *essai_probe* program (function *tbbpool_probe*):

Arena	Largest block expected
256 MiB	about 240 MiB
512 MiB	about 480 MiB
1024 MiB	about 960 MiB

If your measurements are close to these values, the explanation is confirmed.

Limits of this explanation

- *frontend.cpp*, which contains the call to this rounding for user-pool allocations, was not re-read. The link is very probable, but the measurement will settle it.
- No setting that disables this rounding is known.

Practical consequences

- With fixed_pool, plan a margin of 6 to 12.5% on the usable memory.
- For a few large, long-lived arrays, as in your case, this mechanism brings nothing. A pointer (bump) allocator would use 100% of the arena and give strictly deterministic addresses.

Sources

oneTBB documentation, fixed_pool:
https://oneapi-src.github.io/oneTBB/main/reference/scalable_memory_pools/fixed_pool_cls.html
Intel, "How to Use oneTBB for Efficient Memory Allocation in C++ Applications": https://www.intel.com/content/www/us/en/developer/articles/technical/how-to-use-onetbb-for-memory-allocation-cpp.html
tbbmalloc source (2021.5.0): https://docs.rs/crate/tbb-sys/latest/source/tbb/src/tbbmalloc/large_objects.h and backend.cpp (same directory)